



Islamic University of Gaza

Faculty of Engineering

Computer Engineering Department

COSMETICS

PRODUCT MANAGEMENT SYSTEM

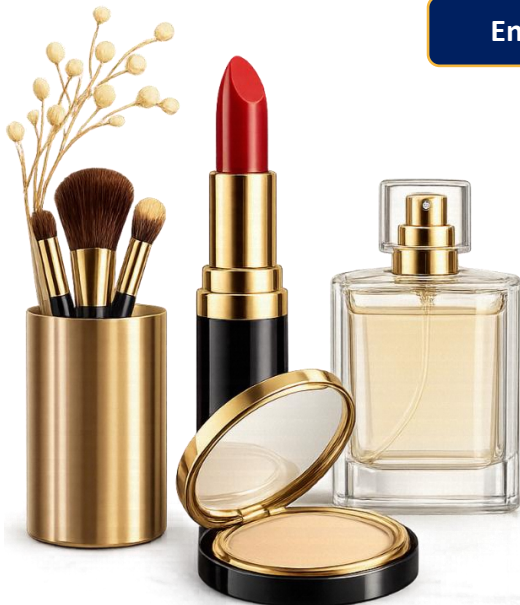
SOFTWARE PROJECT REPORT

Prepared By

Mesk zoheer alkordy	220213008
Aya Rafat Breika	220210776
Hala Awni Ibrahim	220210702
Yara Raafat Abu Taima	220190357
Susan Sameh Qassem	220203034
Shrooq Hassan Musa	220200676
Lama Mahmoud Ruwaished	220212891

Supervisor

Eng. Belal Abu Shawish



Academic year:2025_2026

Gaza,Palestine:May 2026

Introduction

In an era where digital systems are essential for any business, the need for accurate and secure data management has never been greater. The cosmetics industry, in particular, faces unique challenges due to the sheer variety of products, categories (such as skincare, makeup, and

haircare), and the complexity of managing multiple suppliers. Relying on traditional or manual methods is often impractical, leading to data inconsistencies, slow access to information, and a higher risk of errors.

To address these challenges, our project aims to develop the Cosmetics Product Management System (CPMS) using the Laravel framework. The core vision is to transition from unorganized data handling to a modern, structured environment that ensures data integrity and accessibility. The system empowers users with full control over product records—allowing for seamless adding, editing, deleting, and viewing—while maintaining clear relational links between products, their categories, and their suppliers.

Key Pillars of the System:

- **Robust Security:** To protect sensitive information, we are integrating an Authentication and Authorization system. This ensures that only authorized personnel can perform critical actions like editing or deleting, providing a secure and organized environment.
- **Engineering Excellence:** By adopting the MVC (Model-View-Controller) pattern, we ensure a clean separation of concerns, making the application maintainable and ready for future expansion.
- **Dynamic User Experience:** A centralized Dashboard will provide real-time statistics, complemented by advanced features such as search, filtering, and sorting to optimize how users interact with their inventory

Problem Statement

The main challenge in the cosmetics industry today is managing a huge variety of products—ranging from skincare and makeup to perfumes—without a dedicated system. Most small to medium businesses still rely on manual records or unorganized files, which becomes a mess as the inventory grows.

We have identified three core problems that our project aims to solve:

1. **Fragmented Data and Complexity:** Right now, there is no clear link between products, their categories, and their suppliers. When a manager needs to find which supplier provides a specific skincare line, they often have to search through separate, unconnected files, which is slow and frustrating.
2. **Human Error and Inefficiency:** Managing data manually leads to frequent mistakes. Whether it's a typo in a price or accidentally deleting a product record, these errors happen often because there are no validation rules or structured databases to catch them.
3. **Security and Data Risks:** One of the biggest issues is the lack of "who can do what." Currently, anyone can edit or delete important information because there is no system to control user permissions. This lack of authorization makes the data vulnerable to unauthorized changes or accidental loss.

Because of these issues, there is a real need for a system like ours that centralizes everything in one place, automates the links between data, and ensures that only authorized users can make changes.

Project Objectives

The main goal of this project is to build a practical and well-organized system that makes managing cosmetics products much easier than traditional methods. We want to move away from unorganized data and create a reliable digital tool.

Our specific objectives are:

1. **Streamline Product Operations:** We aim to simplify the core processes of adding, editing, deleting, and displaying products. By storing everything in a structured database, we ensure that data is handled better and is always up to date.
2. **Organize Through Relationships:** A key objective is to link products to specific categories (like skincare, makeup, and haircare) and connect them with their respective suppliers. This will make the information clearer and much easier to track .
3. **Enhance Security and Control:** We want to implement a secure authentication and authorization system. This is important to control who can access the system and ensure that only authorized users can perform sensitive actions like editing or deleting data.
4. **Improve Data Visibility and Access:** We aim to provide a user-friendly interface that includes advanced search, filtering, and sorting options. Additionally, we want to create a dashboard that gives a quick overview of the system's status, such as total products and recent additions.
5. **Build a Scalable Foundation:** Beyond just the current features, our objective is to create a solid system using the MVC pattern. This ensures that the project is not just a one-time task, but a scalable application that can be expanded with more advanced features in the future .

1. Requirements

In our team's analysis, we believe that defining clear requirements is the foundation of the entire project. As software engineers, our goal isn't just to write code, but to build a solution that directly addresses the "manual data mess" we identified in our problem statement. We have categorized our requirements into Functional and Non-functional to ensure a high-quality engineering approach.

1.1 Functional Requirements

These requirements represent the core services that our CPMS will provide to solve the daily challenges of cosmetics inventory management:

- **Secure User Authentication:** To protect the shop's data, we decided that the system must allow staff to create accounts and log in securely before accessing any management tools.
- **Complete Product Control (CRUD):** The system shall enable authorized users to perform full operations (Add, View, Edit, and Delete) on cosmetic products to ensure the digital inventory is always up-to-date.
- **Relational Data Linking:** One of our main engineering goals is to ensure that every product is strictly linked to a specific category (Makeup, Skincare) and a specific supplier, which eliminates the fragmented data problem.
- **Live Statistics Dashboard:** We want to provide the manager with a "quick look" at the business status. The system will display real-time numbers of total products, suppliers, and a list of the most recent entries.
- **Smart Search & Advanced Filters:** To save time for the shop staff, we prioritized a search feature that allows finding products by name, and filtering them by category or supplier instantly.
- **Role-Based Authorization:** To prevent accidental data loss, the system must check permissions. For example, while any staff can add a product, only the Manager should have the authority to delete a supplier or a product record.

1.2 Non-Functional Requirements

Beyond what the system does, we focused on "how" it performs to ensure a professional user experience:

- **Robust Data Security:** We prioritized protecting sensitive records. Sensitive actions must be restricted based on user roles to avoid unauthorized changes.
- **High Performance (Speed):** We understand that in a busy shop, time is money. Therefore, our system is designed to process search queries and load the dashboard instantly and without any noticeable delay to ensure a smooth workflow.
- **Usability & Simple UI:** Since the shop staff might not be tech-experts, we are committed to building a clean, simple, and intuitive interface that requires minimal training.
- **System Responsiveness:** We want the shop owner to manage the inventory from any device. The system will be fully responsive, working perfectly on desktops, tablets, and mobile phones.
- **Maintainability & Scalability:** By using the MVC (Model-View-Controller) pattern with Laravel, we ensure that our code is organized. This makes it easy for us (or any other developer) to add new features in the future without breaking the system

2. System Features

In this section, we describe the core functions of our CPMS. As a team, we didn't just want to build a database; we wanted to create a tool that solves real problems like manual errors and fragmented data that we identified earlier

- **Professional Project Identity:** We started by making sure our project has a clear "face." We built a unified flow with Home, About, and Team pages so anyone can understand our vision and see the 8 of us who built this system. We also made sure this interface is fully responsive to work on any device
- **Complete Product Control (CRUD):** To stop the mess of manual records, we built a full management system. We focused on letting the staff add, edit, or delete beauty products easily through digital forms so the stock list is always accurate.
- **Automated Data Integrity:** We didn't want any wrong data in our system, like negative prices or duplicate names. So, we added backend validation that catches these mistakes immediately, ensuring our database stays "clean" and reliable
- **Smart Product Categorization:** We realized that products shouldn't just be a long, confusing list. We linked every item to its specific category (like Makeup or Skincare) so it's much easier for the manager to organize and track the inventory.
- **Advanced Supplier Management:** Dealing with many suppliers is a headache for shop owners. We created a system to link products to them using many-to-many relationships, so we can now track exact costs and delivery times for each supplier in one place
- **Secure Access & Role Permissions:** Security was a top priority for our team. We built a login system with different levels of authority; while staff handle daily entries, only the manager has the authority to perform sensitive actions like deleting records
- **Live Inventory Dashboard:** We wanted a way for the owner to see how the shop is doing at a glance. Our dashboard provides real-time statistics on total products and categories without having to dig into long, boring tables
- **Pro Search & Fast Retrieval:** We know how busy cosmetics shops can get, so we added tools to find any product information instantly. The staff can now search or filter items without any noticeable delay, ensuring they can help customers much faster and improve overall service efficiency

3. User Stories (Agile Analysis)

To ensure our CPMS actually solves the problems of fragmented data and manual errors we identified earlier, our team sat down and analyzed what each user truly needs. We followed the standard Agile format-- As a, I want, So that--because we want our system to be built on real value, just like how professional software companies operate .

3.1 Project Identity & Vision

In our initial analysis, we believed that the first step to building a professional system is establishing a clear digital identity for our project and our team .

- **Story 1: Unified Web Interface & Identity**

- As a new user, I want to browse the basic introductory pages of the system- such as the Home, About Us, and Features pages-so that I can understand the core idea and objectives of the Cosmetics Product Management System (CPMS) before it is developed into a complete, integrated system

3.2 Daily Operations (Data Entry Staff)

We focused here on making the daily work fast and error-free, especially since we found that manual entry was a major cause of data inconsistencies in cosmetics shops .

- **Story 2: Complete Product Control (CRUD)**

- As a Data Entry Staff member, I want to manage product data through a structured database and a full CRUD system, so that the inventory remains accurate and up-to-date while moving away from manual records.

- **Story 3: Data Integrity & Error Prevention**

- As a System Administrator, I want the system to apply strict validation rules (like unique names and positive prices), so that we prevent human errors and ensure all stored data is clean and reliable.

3.3 Organizing the Relationships (Product Manager)

Since our problem statement highlighted a "data mess" due to a lack of links between products, categories, and suppliers, we created these stories to fix that complexity .

- **Story 4: Smart Product Categorization**

- As a Product Manager, I want to link every product to its specific category (Skincare, Makeup), so that the inventory remains organized and I can easily track items by type .

- **Story 5: Advanced Supplier Management**

- As a Procurement Manager, I want to connect products to their respective suppliers using a many-to-many relationship, so that I can track exact sources, costs, and lead times professionally .

3.4 Management & Visibility (Shop Owner)

For the owner, our team prioritized high-level visibility so they can monitor the business status quickly from any device .

- **Story 6: Secure Access & Role Permissions (Security System)**

- As an Administrator, I want to set specific permissions and a secure login gate, so that I can protect our database from unauthorized changes or accidental deletions.

3.5 Search & Efficiency (Sales Team)

- We understand that in a busy shop, time is money Therefore, we prioritized speed in our system design so the staff can find any product information instantly and without any noticeable delay, ensuring a smooth and productive workflow during sales operations.

- **Story 7: Unified Interface & Live Dashboard**

- As a Shop Owner, I want a unified web interface and a live dashboard, so that I can see my shop's status at a glance and navigate between products and suppliers easily.

3.6 Security & Control (Administrator)

We realized that security is a huge risk in manual systems. This story ensures only the right people have the authority to make sensitive changes .

- **Story 8: Fast Search & Retrieval**

- As a Sales Staff member, I want to use an advanced search engine with filters, so that I can find any product details for my customers instantly by their names.

4. System Scenarios

In this section, we describe the detailed "Step-by-Step" flow of how our CPMS works in real life. While our user stories focus on the "what," these 8 scenarios explain the "how" to ensure our system is ready for the development phase

Scenario 1: Secure Admin Access and Dashboard Entry (Mesk)

- **User:** Mesk (Admin).
- **Steps:**
 1. Mesk opens the project URL and the Welcome Page appears, showing the "Log in" and "Register" buttons at the top right corner.
 2. Since the Admin account is already registered and ready, she clicks the "Log in" button to start the authentication process.
 3. The system redirects her to the Login Interface, where she enters the Admin email (admin@gmail.com) and her secure password.
 4. She clicks the "LOG IN" button, and the system validates her credentials instantly and without any noticeable delay .
 5. Once authenticated, Mesk is redirected to the Live Dashboard, giving her full control over the shop's products and categories .

Scenario 2: Adding New Stock

- **User:** Hala (Data Entry Staff).
- **Steps:**
 1. Hala logs in and goes to the "Products" page.
 2. She clicks "Add Product" and fills the form for a "Vitamin C Serum" (Price: \$30).
 3. She clicks "Save," and the system adds the record to the database.
 4. The product appears at the top of the list with a success message .

Scenario 3: Preventing Input Errors

- **User:** Suzan (Data Entry Staff).
- **Steps:**
 1. Susan tries to update a product price but enters a negative number (-\$15) by mistake.
 2. She clicks "Update" to save the change.
 3. The system's Validation Rules block the action and show a red error: "Price must be positive."
 4. The system keeps her old data in the form so she can fix it easily .

Scenario 4: Organizing Items by Category

- **User:** Yara (Product Manager).
- **Steps:**
 1. Yara wants to organize the list by linking products to their types.
 2. She opens the product edit form and picks "Makeup" from the category dropdown.
 3. She clicks "Save," and the system creates a One-to-Many link.
 4. Now, the shop can filter the inventory to see only makeup items .

Scenario 5: Linking Suppliers to Products

- **User:** Shrooq (Product Manager).
- **Steps:**
 1. Shrooq needs to track which supplier provides a specific perfume.
 2. She uses the Many-to-Many interface to link the item to "Paris Beauty Co."
 3. She enters the cost price and clicks "Sync."
 4. The system updates the records, making it easy to track sourcing details.

Scenario 6: Testing User Permissions

- **User:** Nour (Security Administrator).
- **Steps:**
 1. Nour logs in with a "Staff" account to test security limits.
 2. She tries to click the "Delete" button on a product record.
 3. The system hides the button from her view because of the Product Policy.
 4. If she tries to use the delete URL directly, she gets a "403 Forbidden" error .

Scenario 7: Monitoring Shop Stats

- **User:** Lama (Shop Owner).
- **Steps:**
 1. Lama opens the "Live Dashboard" to see how the shop is doing.
 2. She looks at the cards showing the Total Number of Products and Categories.
 3. She checks the "Recently Added" list to see what the staff updated today.
 4. The dashboard gives her a quick summary without searching through tables.

Scenario 8: Fast Search for Customers

- **User:** Aya (Shop Staff).
- **Steps:**
 1. A customer asks Aya for the price of "Organic Hair Oil."
 2. Aya types "Organic" into the Search Bar.
 3. The system filters the records instantly without any noticeable delay.
 4. Aya tells the customer the price and quantity immediately .

5. Specifications (Tasks)

Specifications (Tasks) in Agile refer to the clear, detailed descriptions of what needs to be done in order to fulfill a User Story. They break down the “what” from the User Story into smaller, actionable “how” steps for the development team.

5.1 Specifications for Story 1: Project Identity & Initial Setup (Mesk)

- **User Story:** As a new user, I want to browse the basic introductory pages (Home, About Us, Features) so that I can understand the core idea and objectives of the CPMS project .
- **Tasks:**
 1. Define the web routes within the web.php file to enable navigation between the core pages: Home, About Us, and Features .
 2. Create a HomeController containing several methods to manage the logic for each page and return the appropriate Blade Views .
 3. Implement logic to pass dynamic data from the Controller to the pages using variables and arrays .
 4. Utilize Blade loops within the views to display this passed data dynamically and efficiently.
 5. Establish the basic MVC structure of the project as a foundation for future development, including advanced features like the Dashboard, Product Management, and Authentication/Permissions .

5.2 Specifications for Story 2: Product Model, Migration, Seeder & CRUD Operations (Hala)

- **User Story:** As a Data Entry Staff member, I want to manage product data through a structured database and full CRUD system so that the inventory remains accurate and up.
- **Tasks:**
 1. Create a Product Model in Laravel to represent the products data structure in the application.
 2. Write a Laravel Migration to create the products table with fields (id, name, price, timestamps) in the database.
 3. Define fillable properties in the Product model to allow secure mass assignment of allowed fields.
 4. Create a ProductSeeder to insert sample products into the database for testing and development purposes.
 5. Verify database seeding by checking that sample data is correctly stored in the products table.
 6. Implement full CRUD operations (Create, Read, Update, Delete) for products within the system.
 7. Develop Blade views including index, create, and edit pages to manage product data through the UI.
 8. Implement ProductController methods to handle all CRUD operations and ensure proper data validation and handling.

5.3 Specifications for Story 3: Data Integrity & Error Prevention (Suzan)

- **User Story:** As a System Administrator, I want strict validation rules so that we prevent human errors and ensure reliable data .
- **Tasks:**
 1. Create a Form Request class to centralize backend validation logic for all product entries.
 2. Add validation rules to ensure prices and stock counts are always positive numbers.

3. Implement a unique constraint for Product Names in the database and the form to ensure that every beauty product is unique and avoid any duplicate entries.
4. Display clear red error messages using @error in the UI to guide users when they make mistakes .

5.4 Specifications for Story 4: Smart Product Categorization (Yara)

- **User Story:** As a Product Manager, I want to link every product to its specific category so that the inventory remains organized .
- **Tasks:**
 1. Write a migration to create the categories table and define a One-to-Many relationship with products.
 2. Build a seeder to fill the categories table with items like "Makeup," "Skincare," and "Haircare".
 3. Update the product form to include a dynamic dropdown menu fetched from the categories table.
 4. Ensure the system blocks saving any product that isn't linked to a valid category.

5.5 Specifications for Story 5: Advanced Supplier Management (Shrooq)

- **User Story:** As a Procurement Manager, I want to connect products to multiple suppliers using many-to-many relationships to track sources .
- **Tasks:**
 1. Create a Pivot Table named product_supplier to support Many-to-Many relationships .
 2. Add extra fields to the pivot table to store the specific cost price and delivery time for each supplier.
 3. Use the sync() method in the controller to manage and update supplier associations.
 4. Create a filtered view that lists all products provided by a specific distributor.

5.6 Specifications for Story 6: Secure Access & Role Permissions (Nour)

- **User Story:** As an Administrator, I want to set specific permissions and a secure login gate, so that I can protect our database from unauthorized changes or accidental deletions.
- **Tasks:**
 1. We installed Laravel Breeze to handle registration, login, and logout safely
 2. Product Ownership: We added a user_id to the products table so the system knows "who added what".
 3. We created a Product Policy to make sure only the person who added a product can edit or delete it
 4. We used @can and @if logic in the code to hide the Edit and Delete buttons from anyone who isn't the owner
 5. We protected the website links (routes) so that "guests" can't see the dashboard without logging in first

5.7 Specifications for Story 7: Unified Interface & Live Dashboard (Lama)

- **User Story:** As a Shop Owner, I want a unified web interface and a live dashboard, so that I can see my shop's status at a glance and navigate between products and suppliers easily.
- **Tasks:**
 1. We made a shared file called app.blade.php so that the top and bottom of the site stay the same on every page .
 2. We built a Navbar with links to the Dashboard, Products, and Suppliers so we can move around easily
 3. We created Dashboard Cards that show the total number of products and suppliers we have .
 4. We made a small table on the home page to show the last 5 products we added recently .
 5. We added Flash Messages; a green one for "Success" and a red one for "Error" to tell us what happened

5.8 Specifications for Story 8: Fast Search & Retrieval (Aya)

- **User Story:** As a Sales Staff member, I want to use an advanced search engine with filters, so that I can find any product details for my customers instantly by their names.
- **Tasks:**
 1. We added a "Search Scope" in the Product model to handle queries based on the Product Name.
 2. We wrote code to let users filter the products by their Category or Supplier to find items easily .
 3. We used Laravel Pagination so the product list stays organized and the page loads fast .
 4. We added a message that says "No results found" if the search doesn't match any product name in our database .

6. Acceptance Criteria (Definition of Done)

In this section, we define the specific conditions that each feature must meet to be considered complete and correct. These criteria serve as the basis for validation and final testing .

6.1 Project Identity & Initial Setup (Mesk)

- **Story: Unified Web Interface & Identity**
 1. All links in the top menu (Home, About, Features, and Team) must open the correct pages without any errors.
 2. The "Team" page must clearly show the names and roles of all 8 engineers in our group.
 3. The website's header and footer must stay the same on every page to keep the design organized.
 4. The introductory pages must look clean and be easy to read when opened on a mobile phone or tablet.

6.2 Product Operations (Hala)

- **Story: Complete Product Control (CRUD)**

1. The system must save the product and show a "Success Message" only if all required fields (Name, Price) are valid and formatted correctly .
2. If someone tries to put in a negative price or stock, the system has to block it and show a red warning so we don't save wrong data.
3. As soon as we click save, the new product needs to show up at the very top of our list so we know it was added.
4. When we edit an existing item, the database should update the info correctly without accidentally making a duplicate copy of the same product.

6.3 Data Integrity & Error Prevention (Suzan)

- **Story: Data Integrity & Validation**

1. The system must trigger backend validation to block any attempt to save an empty product name or duplicate product entries, ensuring that every record in our database is unique and accurate.
2. If validation fails, the system must use the old function to keep the user's previous correct inputs in the form fields .
3. Error messages must be specific ("The price field is required") and appear in red near the input field.
4. The database must be strictly set up to never allow "Null" or empty values in mandatory columns like Name and Price to keep our data clean.

6.4 Smart Product Categorization (Yara)

- **Story: Smart Product Categorization**

1. The product form must include a dynamic dropdown menu that fetches the current categories from the database .
2. A product cannot be saved unless it is linked to a valid category (No unorganized items allowed).
3. When a user filters by a category ("Makeup"), the system must accurately display only the products belonging to that category.
4. The category name must be displayed clearly in the main product table for every item

6.5 Advanced Supplier Management (Shrooq)

- **Story: Advanced Supplier Management**

1. The system must support a Many-to-Many relationship, allowing one product to be linked to multiple suppliers .
2. The unique cost price for each supplier must be saved in the pivot table and displayed correctly in the details view .
3. The sync() method must update our supplier list perfectly without causing any database errors or leaving behind "orphaned" (broken) records.
4. When we open a supplier's profile, the system must accurately list every single product that is provided by that specific supplier.

6.6 Secure Access & Role Permissions (Nour)

- **Story: Secure Access & Role Permissions**

1. The system must let users create accounts and log in without any bugs.
2. A user cannot delete or change a product that belongs to someone else.
3. The "Edit" and "Delete" buttons must disappear if the user doesn't own that product.
4. If a user tries to go to a delete link directly, the system must show a "403 Forbidden" error

6.7 Unified Interface & Live Dashboard (Iama)

- **Story: Unified Interface & Live Dashboard**

1. The top menu (Navbar) must work perfectly and take the user to the correct page every time.
2. The website design (Header and Footer) must stay the same no matter which page we open.
3. The numbers in the Dashboard cards must be 100% correct and match our database.
4. The system must show a clear success message after we save any new data .

6.8 Fast Search & Retrieval (Aya)

- **Story: Fast Search & Retrieval**

1. Search results must appear instantly when we type the product name .
2. The system must let us search by Name and filter by Category or Supplier at the same time.
3. If a product name is not in the system, a clear "No results found" message must show up.
4. The results must be divided into Pages to keep the list clean and fast .

7. Software Architecture (Diagrams)

7.1 Use Case Diagram Description

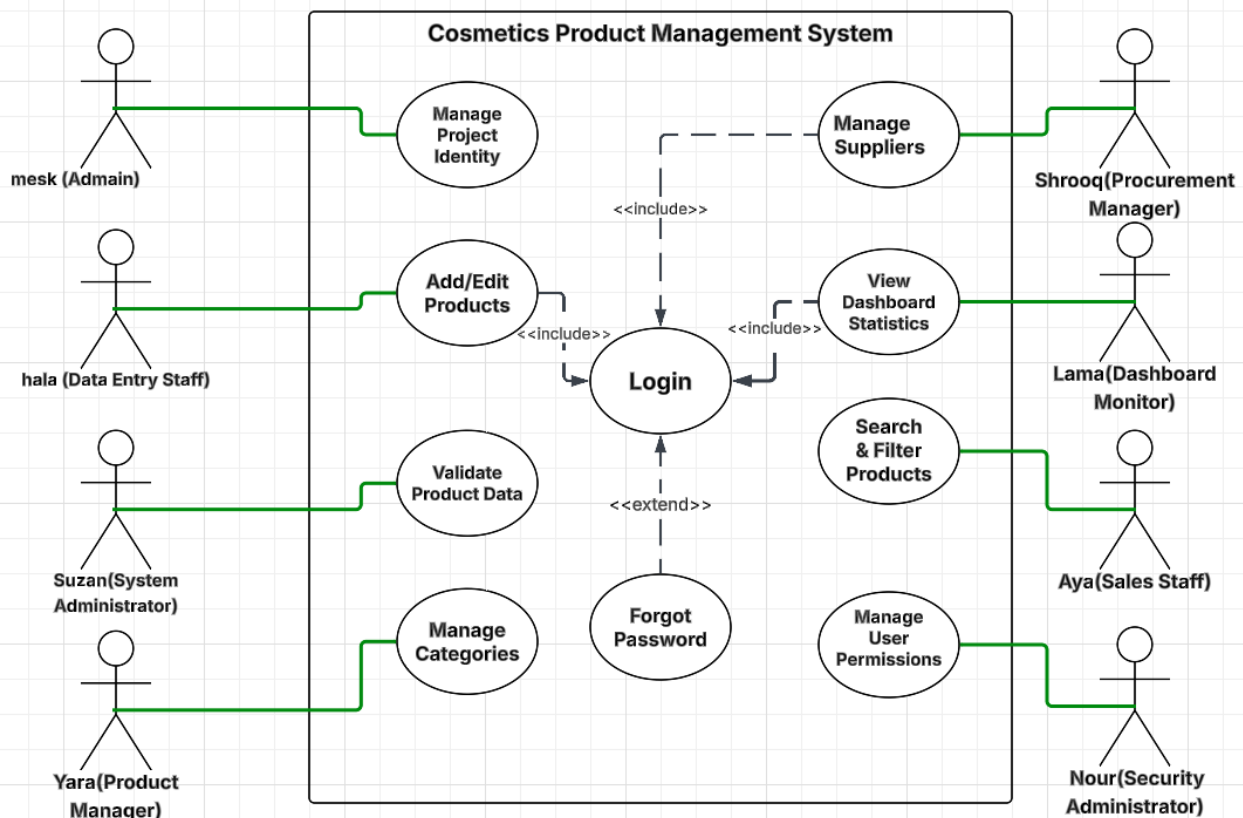
In our Cosmetics Product Management System (CPMS), we designed the Use Case Diagram to visualize how the functional requirements we identified-like CRUD, Search, and the Dashboard-interact with different users. This diagram helps us define the System Boundary and ensure that our internal logic matches the 8 User Stories we developed .

Actors and Their Responsibilities: To ensure "Equal Participation," we assigned each of the 8 engineers in our team a specific role that matches their assigned User Story and Tasks :

- **Mesk (Project Lead):** Handles the Project Identity use cases, ensuring the system vision and team information are clear.
- **Hala (Data Entry Staff):** Responsible for the Complete Product Control (CRUD) use case to manage daily inventory .
- **Suzan (System Administrator):** Focuses on the Data Integrity logic within the use cases to prevent input errors .
- **Yara (Product Manager):** Manages the Smart Categorization use cases to keep products organized by type .
- **Shrooq (Procurement Manager):** Handles the Advanced Supplier Management use case, linking products to their sources .
- **Lama (Shop Owner):** Interacts with the Live Inventory Dashboard to monitor real-time business statistics .
- **Aya (Sales Staff):** Uses the Fast Search & Filter use case to retrieve product details for customers instantly and without any noticeable delay, ensuring a fast and efficient shopping experience.
- **Nour (Security Administrator):** Manages the Secure Access & Permissions use case to protect the database .

System Logic & Security Relationships: To satisfy our Acceptance Criteria for security and error prevention, we implemented specific engineering links in the diagram:

- **<<include>> Relationships:** We linked sensitive use cases (Manage Products, View Dashboard, and Manage Suppliers) to the Login & Logout function. This means the system "Definition of Done" for these features requires a verified user session in our Laravel backend .
- **<<extend>> Relationship:** We added Forgot Password as an extension of the Login use case. This provides an optional flow that meets our non-functional requirement for usability (UX).



7.2 Deployment Diagram Description

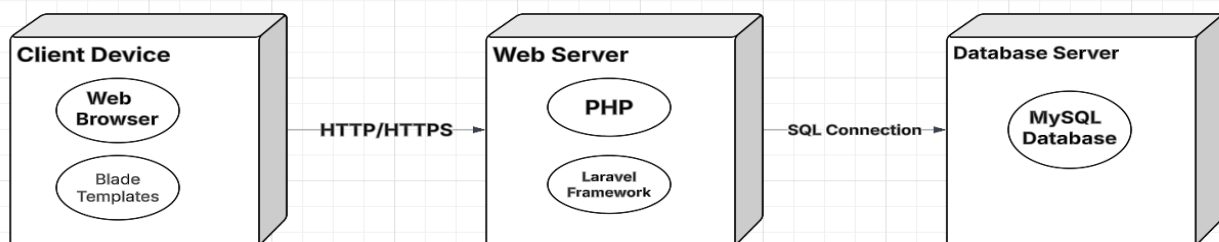
This diagram illustrates the physical architecture of our Cosmetics Product Management System (CPMS). It shows how our software components are distributed across different hardware nodes to ensure the security, speed, and reliability required for a professional inventory tool.

System Nodes and Components:

- 1. Client Device:** This represents the computers or mobile devices used by our team members (such as Aya and Hala) to access the system. The interaction happens through a Web Browser, displaying the Blade interfaces we designed to be fully responsive for both desktop and mobile views.
- 2. Web Server:** This is the "brain" of our project where the Laravel Framework and PHP logic are hosted. It manages the MVC pattern, handles all user requests, and enforces the Security Policies we implemented to protect the database from unauthorized changes.
- 3. Database Server:** A dedicated node for our MySQL Database. It securely stores all the data related to cosmetics products, suppliers, and categories. This separation ensures that complex queries, such as our advanced search and Many-to-Many supplier links, are processed instantly and without any noticeable delay, meeting our high-performance requirements.

Communication Protocols:

- **HTTP/HTTPS:** This protocol is used for secure communication between the Client Device and the Web Server, ensuring that data transferred over the web remains protected.
- **SQL Connection:** The Web Server communicates with the Database Server using a secure SQL connection to perform the CRUD operations and data retrieval tasks we developed in our project.

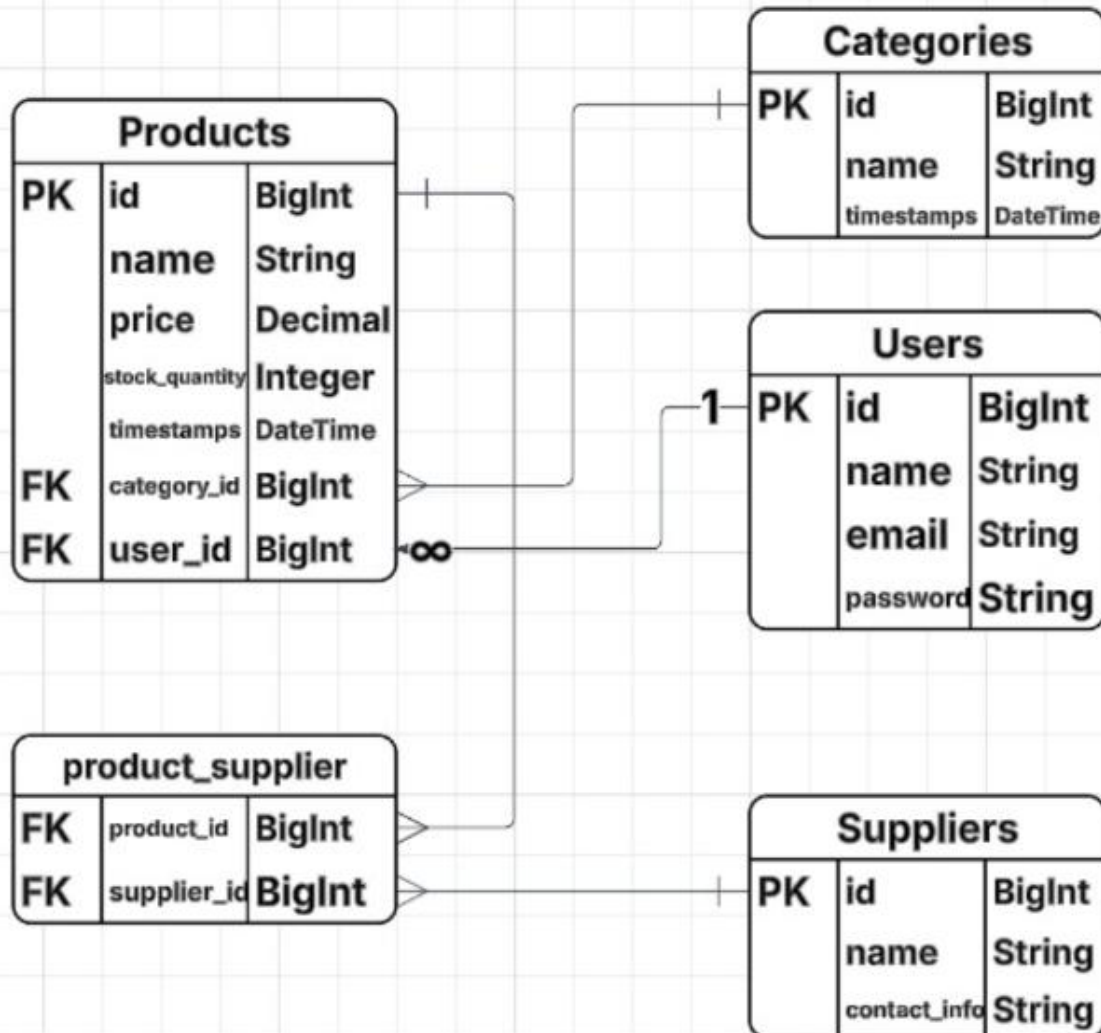


7.3 Entity-Relationship Diagram (ERD) Description

As a team, we believe that the database is the most critical part of our CPMS project. Our main goal was to move away from the "manual data mess" and build a professional system that is "founded correctly" This ERD shows how we organized our data logic to ensure it is secure, connected, and ready for future growth.

Our Engineering Design Choices:

- **Centralized Product Management:** We placed the Products table at the heart of our system. We chose the Decimal data type for prices to ensure financial accuracy and added Timestamps to every record so we can track exactly when inventory is updated.
- **Smart Categorization (Story 4):** To keep the shop organized, we established a One-to-Many relationship between Categories and Products. This means every beauty item, whether it's Skincare or Makeup, has a clear home, making it easy for the manager to filter the inventory .
- **Advanced Sourcing (Story 5):** We knew that managing multiple vendors is a big challenge. So, we designed a Many-to-Many relationship using a pivot table called product_supplier. This allows us to link products to multiple suppliers and track sourcing details professionally .
- **Secure Ownership (Story 6):** Following our "Robust Security" pillar, we linked the Users table to Products. This is how we implemented the Product Policy; each item "knows" who added it, ensuring that only authorized staff can edit or delete records .



8. Software Testing Cases: Specification and Execution

Test ID	Responsible Engineer	User Story	Test Steps	Expected Result
01	Mesk	Project Identity & Routing	Open the project URL and click on "About Us" and "Team" links.	All links must redirect correctly to their respective pages without any 404 errors
02	Hala	Product CRUD Operations	Fill the "Add Product" form with "Moisturizer", Price \$25, and click Save.	The system saves the record and displays a green success message at the top
03	Susan	Data Integrity & Validation	Try to update a product price to a negative value (- \$10) or leave the Name field empty.	System blocks the action and displays a red error message: "The name field is required"
04	Yara	Smart Categorization	Assign a product to the "Skincare" category and use the category filter.	The system correctly links the product (One-to-Many) and displays it under the selected filter
05	Shrooq	Supplier Management	Link a product to "Global Beauty Co" using the Many-to-Many interface.	Pivot table updates successfully with the specific supplier cost and delivery time
06	Nour	Security & Permissions	Log in as a "Staff" member and try to delete a product added by the "Admin".	The Delete button is hidden, or the system returns a "403 Forbidden" error if accessed via URL
07	Lama	Live Inventory Dashboard	Open the Dashboard and check the "Total Products" count.	Numbers on dashboard cards must match the actual records in the database precisely
08	Aya	Advanced Search & Retrieval	Type "Organic" in the search bar and press Enter.	Matching results appear instantly without any noticeable delay

9. Conclusion

In conclusion, the Cosmetics Product Management System (CPMS) represents more than just a software application; it is the culmination of a rigorous Software Engineering journey conducted under the Agile methodology. Throughout this project, our team of seven engineers has successfully transitioned from theoretical concepts to a practical, structured environment that solves real-world challenges such as fragmented data, human errors, and security risks in the cosmetics industry.

By adopting the MVC (Model-View-Controller) pattern and implementing robust Authentication and Authorization systems, we have ensured that our system is not only functional but also secure, maintainable, and scalable for future expansions. The integration of professional engineering tools like GitHub for version control and Jira for task management allowed us to simulate a real-world software development environment, fostering "Equal Participation" and effective team collaboration.

Ultimately, this project has redefined our perspective as Software Engineers, proving that excellence lies in the balance between deep system analysis, precise architectural design, and thorough Software Testing. We believe that CPMS stands as a professional digital tool that meets high-quality engineering standards and fulfills the strategic objectives of modern inventory management.

The End